

Speaker Notes

Potential-Based Reward Shaping for Planar Pushing Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime (s3426394)

June 2026 — University of Groningen, FSE

This handout reproduces the slide speaker notes verbatim. Slides that contain mathematics also carry a *Formula breakdown* defining every symbol and stating what the maths means.

Slide 1 — Title Page

- Welcome — 30-minute presentation on my master thesis
- Topic: using potential-based reward shaping to train a UR5e robot arm to push objects to goal poses
- I built four model variants and systematically compared them
- Key finding: the simplest model wins — PPO-PBRS (single-agent) achieves 80% validation success
- Curriculum is competitive but adds no value at scale; adversarial self-play makes things worse
- I'll explain why and what this means for RL in contact-rich manipulation

Slide 2 — Overview: Thesis Structure

- This is the roadmap for the whole talk — four stages, left to right
- Stage 1, Planar Pushing MDP: I define the task, the quasi-static push mechanics (limit surface), and the SE(2) distance metric
- Stage 2, PBRS Theory: why the old ad-hoc reward failed, then the potential-based reward shaping theory and our potential functions
- Stage 3, Four Model Variants: PPO-PBRS (single-agent, our best), PPO-Curriculum, and two self-play variants (ASP-dPose, TASP-dPose)
- Stage 4, Key Findings: why ASP fails, the central independence result, computational overhead, and the overall comparison and conclusions
- Next slide states the three research questions that this roadmap answers

Slide 3 — Research Questions

- Three questions structure the whole presentation
- RQ1: PPO-PBRS achieves 80% validation SR; compared head-to-head against the hand-tuned (classic) dense reward on the same protocol
- RQ2: No — PPO-Curriculum is statistically tied and self-play (ASP-dPose, TASP-dPose) underperforms the simple single-agent

- RQ3: A sufficiently informative reward (PBRs) substitutes for an explicit curriculum at sufficient batch size; at small batch the curriculum helps
- Roadmap: I'll define the MDP, explain why PBRs was needed, walk through each model, and present the comparison
- Four models, 3000 iterations each, 528 parallel environments on an RTX Pro 6000

Slide 4 — MDP Formulation: State & Action

- The MDP is a planar pushing task: a UR5e robot pushes a T-block object on a $1.4\text{m} \times 1.0\text{m}$ table
- State is 28D: end-effector pose (6), object state including position and quaternion (14), goal pose (6), and current distance errors (2)
- Action is 4D macro-parameterized as object-relative approach (r, ϕ) plus world-frame push direction (length, θ). 21 bins per dimension = 194k discrete actions
- Object-relative actions are critical — they guarantee 95% contact rate regardless of object spawn position, vs 2% for absolute coordinates

Formula breakdown

$$\mathbf{s} = [\mathbf{q}_{ee}(6) \mid \mathbf{p}_{obj}, \mathbf{R}_{obj}(14) \mid \mathbf{p}_{goal}, \mathbf{R}_{goal}(6) \mid d_{pos}, d_{rot}(2)] \in \mathbb{R}^{28}$$

Terms:

- $\mathbf{q}_{ee}$ — end-effector pose (position + orientation), 6 numbers
- $\mathbf{p}_{obj}, \mathbf{R}_{obj}$ — object position and orientation state, 14 numbers
- $\mathbf{p}_{goal}, \mathbf{R}_{goal}$ — target pose the object must reach, 6 numbers
- d_{pos}, d_{rot} — current position and rotation error to the goal, 2 numbers

Meaning: the complete observation the policy reads each step — everything needed to choose the next push.

$$\mathbf{a} = (r, \phi, \ell, \theta), \quad 21^4 = 194,481 \text{ discrete actions}$$

$$X_s = x_{obj} + r \cos(\psi_{obj} + \phi), \quad Y_s = y_{obj} + r \sin(\psi_{obj} + \phi), \quad X_f = X_s + \ell \cos \theta, \quad Y_f = Y_s + \ell \sin \theta$$

Terms:

- r — approach radius from the object centre (where to touch); ϕ — approach angle in the object frame
- ℓ — push length; θ — push direction in the world frame
- ψ_{obj} — object yaw; (X_s, Y_s) push start, (X_f, Y_f) push end; 21 bins per dimension

Meaning: one push is a macro-action: pick a contact point on the object (r, ϕ) , then shove it a distance ℓ in direction θ . Defining it relative to the object guarantees the gripper actually contacts it.

Slide 5 — MDP Formulation: Transition & Episode

- The physics is quasi-static: the object motion is determined by the limit surface normality theorem — every push couples translation and rotation
- Episode runs max 15 pushes. Terminates on combined success ($\text{pos} < 5\text{cm}$ AND $\text{rot} < 0.2 \text{ rad}$) or on catastrophe (tipped, launched, out-of-workspace)

- The table is $1.40\text{m} \times 1.00\text{m}$, IK workspace $X \in [-0.50, 0.50]$, $Y \in [0.25, 0.70]$
- GIF suggestion shows a real robot doing 3 pushes to reach a goal — this grounds the abstract MDP in the physical task

Formula breakdown

$$\mathbf{t} = (v_x, v_y, \omega) \propto \nabla f(\mathbf{w})$$

Terms:

- $\mathbf{t}$ — the object twist: planar linear velocity (v_x, v_y) plus angular velocity ω
- $f(\mathbf{w})$ — the limit surface, the boundary in wrench space $\mathbf{w} = (\text{force}, \text{torque})$ the contact can sustain
- ∇f — gradient (the surface normal direction)

Meaning: the resulting motion points along the surface normal, so a single push almost always produces translation *and* rotation at once — you cannot command one without the other.

Slide 6 — Push Mechanics: Why Position & Rotation Are Coupled

- The central physics insight: in quasi-static planar pushing, translation and rotation are mechanically coupled through the limit surface
- The limit surface is a convex hull in wrench space — all force-torque combinations the contact patch can support
- Because the surface is convex and smooth, the motion direction (twist) is always normal to the surface at the applied wrench point
- This means no "push forward without rotating" or "rotate without translating" — every push is a coupled SE(2) motion
- The characteristic length L is the radius of gyration of the pressure distribution. For the T-block $L=0.07\text{m}$ (rotation strongly couples to translation)

Formula breakdown

$$\mathbf{t} \propto \nabla f(\mathbf{w}), \quad L = \text{radius of gyration}$$

Terms:

- $\mathbf{t} \propto \nabla f(\mathbf{w})$ — normality law (same as previous slide): twist is normal to the limit surface
- L — characteristic length / radius of gyration of the object's pressure distribution (metres)
- T-block $L = 0.07\text{m}$: rotation strongly couples to translation through the limit surface

Meaning: L quantifies *how much* a push rotates the object; it is fixed by the object's mass/contact geometry, not chosen — which is why the reward can use it as a physical constant rather than a tuned knob.

Slide 7 — SE(2) Unified Distance (Self-Play Variants)

- The SE(2) unified distance replaces separate position and rotation errors with a single Riemannian metric on the planar rigid-body configuration space
- L converts radians to meters. For the T-block with $L=0.07\text{m}$, 0.5 rad of rotation contributes 3.5cm to d_{pose} — roughly equal weight with a 3.5cm position displacement

- The single-potential PBRS eliminates the competing gradient problem: separate pos/rot potentials each pull in different directions, while the SE(2) potential provides a unified gradient
- ASP-dPose (SE(2) self-play) collapses just like separate-potential self-play — the SE(2) metric doesn't rescue the self-play dynamics, confirming the curriculum is the root cause, not the reward metric

Formula breakdown

$$d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2} \quad \Phi(s) = w e^{-k d_{\text{pose}}^2}, \quad k = 30, \quad w = 10$$

Terms:

- dx, dy — object→goal position error in x, y (metres); $d\theta$ — yaw error (radians)
- L — characteristic length (metres), converts the radian error into metres so it can be added to dx, dy
- d_{pose} — single SE(2) distance to the goal (metres)
- $\Phi(s)$ — potential (state "goodness"); w — peak height; k — decay sharpness

Meaning: d_{pose} is one scalar that fuses position and orientation error, with L as the exchange rate between rotating and translating; Φ turns that distance into a smooth bump peaking at the goal, so position and rotation are optimised by one gradient instead of two competing ones.

Slide 8 — Old Reward Formula: Why It Failed

- The old formula had three fatal flaws that collectively explain why the hand-tuned (classic) dense-reward baseline plateaued around 17% training SR
- First, it's non-Markov: dividing by d_{prev} means reward depends not just on s and s' but on the full history. The same state transition gets different rewards from different starting positions. This violates the Markov property that underlies all RL theory. In my thesis appendix, I prove that no static potential function $\Phi(s)$ can reproduce this reward
- Second, gradient starvation: a push that genuinely produces progress (1cm closer from 30cm away) gets penalized because the penalty term $\beta \cdot d_{\text{now}}$ dominates the tiny improvement term. The expected reward for random pushes is negative with near-zero variance — PPO's GAE advantage estimator gets literally no signal to differentiate good from bad pushes
- Third, in earlier versions (before P63), independent $\alpha_{\text{pos}}=12$ and $\alpha_{\text{rot}}=1$ caused rotation gradients to dominate position $20\times$, resulting in $\text{RotSR}=42\%$ while PosErr stayed flat at 0.25m — the agent learned to rotate but never learned to translate
- The end result: 17.3% training SR, 0.78 rad rot error (essentially random rotation). This is WHY we needed a better reward function

Formula breakdown

$$R = \alpha \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}} + \alpha \frac{y_{\text{prev}} - y_{\text{now}}}{y_{\text{prev}}} - \beta d_{\text{now}} - \beta_{\text{rot}} y_{\text{now}}, \quad \alpha = 3.0, \quad \beta = 0.5, \quad \beta_{\text{rot}} = 0.25$$

Terms:

- d — position error, y — rotation error; subscripts prev/now = before / after the push
- α — gain on fractional improvement; $\beta, \beta_{\text{rot}}$ — penalties on the error still remaining

Meaning: it pays for *fractional* progress minus a standing penalty — but dividing by d_{prev} makes the same transition worth different amounts depending on where you started (non-Markov), and the penalty term swamps small honest gains.

$$\underbrace{3 \cdot \frac{0.01}{0.29}}_{\text{improvement}} - \underbrace{0.5 \cdot 0.29}_{\text{penalty}} = -0.025, \quad \mathbb{E}[R] \approx -0.15$$

Meaning: a genuinely good 1 cm push from 30 cm away scores *negative*; random pushes average ≈ -0.15 with almost no variance, so PPO’s advantage estimator sees no signal to tell good pushes from bad — the agent never learns.

Slide 9 — PBRS: The Shaping Theorem

- The fundamental theorem: add any potential-based shaping $F = \gamma \Phi(s') - \Phi(s)$ without changing the set of optimal policies
- Proof: $Q'(s,a) = Q(s,a) - \Phi(s)$. Since $\Phi(s)$ depends only on state (not action), subtracting it from Q doesn’t change which action maximizes Q
- Episodic PBRS (Grzeş 2017): $\gamma_{\text{shaping}}=1.0$ is valid because the sum over an episode is bounded
- This decoupling of reward from policy is the core insight — you can design rewards purely for learning speed without worrying about distorting the objective

Formula breakdown

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \gamma \Phi(s') - \Phi(s) \quad \implies \quad Q'(s, a) = Q(s, a) - \Phi(s)$$

Terms:

- $\mathcal{R}$ — original (true) reward; $\mathcal{R}'$ — shaped reward the agent actually trains on
- γ — discount factor; $\Phi(s)$ — any function of the state alone (the potential)
- Q, Q' — action-values under the original / shaped reward

Meaning: adding exactly $\gamma \Phi(s') - \Phi(s)$ shifts every action’s value by $-\Phi(s)$, which does not depend on the action a ; so the best action (the $\arg \max$) is unchanged. The reward can be redesigned to speed up learning while provably keeping the same optimal policy.

$$\sum_t (\gamma \Phi(s_{t+1}) - \Phi(s_t)) = \Phi(s_T) - \Phi(s_0) \quad (\text{episodic, } \gamma_{\text{shaping}} = 1)$$

Meaning: over a complete episode the shaping telescopes to a fixed constant, so it is bounded and cannot be “farmed” — which is why $\gamma_{\text{shaping}} = 1$ is safe for our finite-horizon task.

Slide 10 — Why PBRS Works for Planar Pushing

- Six properties: policy invariance means you don’t have to worry about the shaping distorting the optimal behavior
- Markovian: the reward depends only on s and s' , not on history. Old formula violated this by dividing by d_{prev}
- Zero-sum cycles: detour pushes produce zero net F — the agent isn’t penalized for exploratory movements
- No constant drain: unlike the old formula which had a $\beta \cdot d_{\text{now}}$ penalty, if the agent doesn’t move, $F=0$

Formula breakdown

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

Terms:

- F — the dense shaping reward added on every push; $\Phi(s')$, $\Phi(s)$ — potential after / before the push

Meaning: the reward depends only on the change in potential between two states — hence Markovian, exactly zero around any loop (no reward hacking), and exactly zero when nothing moves (no penalty for thinking).

Slide 11 — Why PBRS Works: The Reward Diet

- The reward diet plot shows the agent gets 85% of its reward from dense PBRS shaping, with sparse bonuses and penalties as supplementary signals
- Because the dense signal fires every push and is bounded, the critic stays stable and PPO always has a gradient to follow

Slide 12 — PBRS: Our Potential Functions

- We use two potential variants: separate position/rotation potentials (PPO-PBRS, PPO-Curriculum) and the unified SE(2) potential (the self-play variants)
- The separate variant uses exponential potentials: $k_p=30$ makes the potential decay sharply — near the goal the gradient is strong, far away it's weak
- Cosine angular distance $c(s) \in [0,1]$ is chosen over raw yaw error because it's smooth and bounded — no discontinuity at $\pm\pi$
- Dense reward is purely difference-based: $F = \Phi(s') - \Phi(s)$. Sparse completion bonuses (+5 pos, +2 both-threshold) fire on the final push

Formula breakdown

$$\Phi(s) = w_{\text{pos}} e^{-k_p d_{\text{pos}}^2} + w_{\text{rot}} e^{-k_r c(s)}, \quad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2} \in [0, 1]$$
$$k_p = 30, \quad k_r = 5, \quad w_{\text{pos}} = w_{\text{rot}} = 10.0$$

Terms:

- two bumps: one for position (d_{pos}), one for rotation ($c(s)$)
- $c(s)$ — cosine angular distance; θ^* — goal yaw, θ — current yaw
- k_p, k_r — decay sharpness of each bump; $w_{\text{pos}}, w_{\text{rot}}$ — their weights

Meaning: two independent smooth potentials reward getting closer in position and in orientation; the cosine form keeps the rotation term continuous across the $\pm\pi$ wrap-around, so there is no artificial cliff at 180° .

Slide 13 — PBRS: Why Our Potentials Work

- Bounded exponential potentials prevent reward explosions from PhysX glitches; the potential difference gives a natural gradient toward the goal
- Cosine distance respects the S^1 topology of rotation

- These hyperparameters are robust: 3 independent training runs with identical settings all converged to similar performance

Formula breakdown

$$F = \Phi(s') - \Phi(s), \quad 0 \leq e^{-k d^2} \leq 1$$

Meaning: because each exponential is bounded in $[0, 1]$, the per-step reward is bounded too — physics spikes cannot blow up the return — while $\Phi(s') - \Phi(s)$ still points uphill toward the goal. The cosine term treats yaw as living on a circle (S^1), so $-\pi$ and $+\pi$ are the same point.

Slide 14 — PPO-PBRS: Single-Agent (Our Best Model)

- PPO-PBRS is the simplest possible architecture: single PPO agent with LSTM and MLP, PBRS reward, sparse completion bonuses
- Three reasons it works: (1) PBRS provides dense gradient at every push — unlike self-play where sparse bonuses fire on a tiny fraction of pushes, (2) fixed random goal distribution — no adversarial shift breaking PPO, (3) single network — no multi-agent optimization instability
- Converges within 1500 iterations. Stable value function.

Formula breakdown

LSTM($28 \rightarrow 256$)+MLP[512, 256], head 4×21 ; $\gamma = 0.998$, $\lambda = 0.95$, $\varepsilon = 0.2$, ent_coef = 0.005

Terms:

- $28 \rightarrow 256$ — 28-D observation encoded into a 256-D recurrent (LSTM) state; MLP widths 512, 256
- head 4×21 — the 4 action dimensions, each a 21-way categorical
- γ — discount (near 1 = long horizon); λ — GAE smoothing; ε — PPO clip width; ent_coef — entropy-bonus weight

Meaning: a standard recurrent PPO agent; $\gamma = 0.998$ values the whole episode, $\varepsilon = 0.2$ sets the trust-region size, and a small entropy coefficient gives mild exploration without drowning the gradient.

Slide 15 — PPO-PBRS: Training Dynamics

- Left plot: training success rate over 3000 iterations. PPO-PBRS (blue) steadily improves from near-zero to 9.4% (strict combined pos+rot gate). PPO-Curriculum follows a similar trajectory; self-play variants plateau at near-zero
- Right plot: value loss stays stable at 8-12 — no explosion. Earlier attempts with gain=1.0 on the critic caused Val loss 27k-356k (Fix P29). Gain=0.01 keeps the critic stable
- The PBRS exponential potentials prevent the reward spikes that caused GAE chain reactions and critic instability

Formula breakdown

$$\text{critic output gain} = 0.01 \Rightarrow V_{\text{init}} \approx 0.057$$

Terms:

- gain — the initial scale of the critic’s final linear layer; V_{init} — its value prediction at iteration 0

Meaning: a tiny initial gain keeps the critic’s first guesses near zero, so the GAE return estimates start small and bounded instead of exploding into the hundreds of thousands (the failure seen with gain = 1.0).

Slide 16 — PPO-PBRs: Validation Results

- Validation: 30 held-out T-block scenes, 20 trials per scene (up to 30 pushes each). 80% scene SR
- Perfect on position-only scenes (100%), strong on the hardest pos+rot scenes (70%)
- PosErr 0.032m and RotErr 0.568 rad are averaged over all 30 scenes incl. failures; on successful scenes PosErr is 0.010m
- Per-attempt (trial-level) SR is 66% (396/600); scene SR counts a scene solved if any of its 20 trials passes
- The overhead clip is the trained PPO-PBRs policy recorded top-down (record_push_video.py)

Slide 17 — PPO-PBRs: Validation Runs (overhead)

- Three representative validation runs recorded top-down with record_push_video.py
- Left: a short forward push (E_Forward, position-only)
- Middle: a lateral push (E_Left, position-only)
- Right: a combined translate-and-rotate scene (E_Diag, position+rotation)
- All use the same simple PPO-PBRs checkpoint with object-relative obs+actions

Slide 18 — PPO-PBRs: Validation Breakdown

(no speaker notes yet)

Slide 19 — PPO-Curriculum: Forced pos→rot Curriculum (Fixed)

- PPO-Curriculum was designed to learn position first, then progressively add rotation — a classic curriculum approach
- The original trigger was mis-specified: it keyed on a per-push mean position error below 0.08m, a threshold below even the best model’s validation PosErr — unreachable by construction, so the curriculum never activated (this was the P82 bug)
- Corrected trigger: gate on episodic position-only SR ≥ 0.5 over a 20-iter lookback. The curriculum now activates and ramps in the rotation objective
- Result: PPO-Curriculum reaches 76.7% SR — only 3.3pp below PPO-PBRs, and statistically tied. A good reward makes the staged curriculum redundant at this scale

Formula breakdown

original (broken): mean per-push PosErr < 0.08 m; corrected: episodic pos-only SR ≥ 0.5 for 20 iters, w_{rot}

Terms:

- per-push mean PosErr — average object→goal distance over every push in an iteration (dominated by early, far-from-goal pushes; has a structural floor well above 0.08 m)
- episodic pos-only SR — fraction of episodes that reach the position gate (a true “position is solved” signal)
- w_{rot} — weight on the rotation potential; 0 = position-only, 10 = full rotation objective

Meaning: the first trigger watched the wrong quantity (a per-push mean with a floor above its own threshold) so Phase 2 never fired; keying instead on episodic position success makes the curriculum activate — but the resulting model still only ties the no-curriculum baseline.

Slide 20 — PPO-Curriculum: Results

- On the identical 30-scene protocol: PPO-Curriculum 76.7% vs PPO-PBRS 80.0% — a 3.3pp gap that is NOT statistically significant (SE of difference $\pm 10.6\text{pp}$)
- PPO-Curriculum actually has tighter position precision (0.023 vs 0.032 m) from the pos-only Phase 1, but slightly worse rotation (0.663 vs 0.568 rad); both reach 100% pos-only, 65% vs 70% pos+rot
- This is the core RQ3 evidence: a sufficiently informative reward (PBRS) substitutes for the explicit curriculum at this batch size — the curriculum is competent but adds no value

Slide 21 — Self-Play: ASP / TASP Architecture (Alice/Bob)

- The self-play architecture: Alice (goal proposer) and Bob (goal achiever) play an adversarial game. Alice gets +5 when Bob fails, -1 when Bob succeeds (outcome-based)
- Bob sees Alice’s final object pose as his goal. He gets PBRS dense reward + sparse completion bonuses
- When Bob fails, Alice’s trajectory is stored in an ABC buffer for behavioral cloning with $\beta_{\text{ABC}}=0.5$ (outcome-based variant only)
- The two evaluated variants differ only in Alice’s reward: ASP-dPose (outcome-based) and TASP-dPose (time-based); both use the SE(2) d_pose metric on the T-block. Base ASP and the GoalEncoder-ablation are excluded — no matched-protocol Isaac validation

Formula breakdown

$$R_A = \begin{cases} +5 & \text{Bob fails} \\ -1 & \text{Bob succeeds} \end{cases} \quad R_B = \underbrace{F = \Phi(s') - \Phi(s)}_{\text{dense PBRS}} + \underbrace{(+5/-2)}_{\text{sparse}}, \quad \beta_{\text{ABC}} = 0.5$$

Terms:

- R_A — Alice’s (proposer’s) reward; R_B — Bob’s (achiever’s) reward
- β_{ABC} — weight of the Alice-Behavioral-Cloning loss that imitates Alice’s trajectory on Bob’s failures

Meaning: Alice is paid to pose goals Bob cannot reach and Bob is paid to reach them — an adversarial game intended to auto-generate a curriculum; β controls how strongly Bob copies Alice when he fails.

Slide 22 — Self-Play Diagnostic: Alice vs. Bob

- Alice proposes valid, low-displacement goals (mean displacement 0.17m) — she is not posing impossible goals
- Yet Bob rarely satisfies the combined pos+rot gate under the non-stationary adversarial goal distribution
- Per-axis training metrics (position and rotation looked at separately) are NOT independent Bernoulli trials — the combined gate is the only valid success measure, so the apparent “high per-axis, near-zero combined” is largely a gate/measurement artifact, not proof of a toxic curriculum
- Time-based self-play (TASP-dPose) partially rescues this: 16.7% vs 6.7% — suggesting the time-based Alice reward self-regulates toward solvable goals

Slide 23 — Self-Play Variants: ASP-dPose vs TASP-dPose

- Two self-play variants are evaluated, both on the T-block with the SE(2) d_{pose} Bob reward; they differ only in Alice’s reward
- ASP-dPose: outcome-based Alice (+5 on Bob fail / -1 on success), ABC enabled — physics-aligned single SE(2) potential, eliminates competing gradients
- TASP-dPose: Sukhbaatar’s time-based Alice reward, a self-regulating curriculum that rewards Alice for goals Bob takes longer to solve; ABC disabled
- The hypothesis was that either change might rescue self-play — neither closes the gap to single-agent

Formula breakdown

ASP-dPose: $\Phi(s) = w e^{-k d_{\text{pose}}^2}$

TASP-dPose: $R_A = \gamma_{sp} \max(0, t_B - t_A), \quad \gamma_{sp} = 0.5$

Terms:

- Φ with d_{pose} — the single SE(2) potential from Slide 7 (replaces the two competing potentials)
- R_A — Alice’s time-based reward; t_A — pushes Alice used; t_B — pushes Bob used (or the cap on failure); γ_{sp} — scale

Meaning: both share the physics-aligned SE(2) Bob reward; the only difference is Alice’s incentive — outcome-based (pay Alice when Bob fails) vs time-based (pay Alice for the effort gap $t_B - t_A$, a self-regulating curriculum).

Slide 24 — Self-Play Variants: Results

- ASP-dPose reaches 6.7% validation SR; TASP-dPose reaches 16.7% — both far below PPO-PBRS’s 80% (4.8–11.9× gap)
- The combined pos+rot gate is the bottleneck: no self-play variant exceeds 10% pos+rot, and both have $\text{RotErr} > 1.4$ rad — rotation is essentially never learned
- Time-based self-play (TASP-dPose, 16.7%) beats outcome-based (ASP-dPose, 6.7%) by 10pp — Sukhbaatar’s self-regulation partially corrects the adversarial curriculum by rewarding Alice for solvable goals
- But even time-based self-play can’t rescue the structural failure modes: non-stationary goal distribution + sparse-reward starvation + (for the outcome variant) vanishing ABC gradients

Formula breakdown

$$R_A \propto \max(0, t_B - t_A)$$

Meaning: Alice earns more the longer Bob struggles (t_B large) relative to her own effort (t_A small); in principle this self-limits goal difficulty, and in practice it lifts results from 6.7% to 16.7% — but the structural failures still cap performance far below single-agent.

Slide 25 — Why ASP Fails: Failure Modes 1 & 2

- The three failure modes form a vicious cycle. The first two are on this slide
- First, non-stationary goal distribution: Alice learns continuously, so Bob’s training distribution shifts every iteration. PPO computes the importance ratio using `old_logp` from a rollout collected under a DIFFERENT Alice. The ratio is dominated by Alice’s distribution shift, not Bob’s weight updates. PPO’s clip fires on ~80% of transitions, killing the gradient
- Second, sparse reward starvation: Bob’s sparse bonuses (+1/-1/+5) fire on 0.14% of pushes. GAE advantages = zero-mean noise. The agent cannot distinguish good pushes from bad pushes

Formula breakdown

$$\text{ratio} = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)} \text{ clipped to } [1 - \varepsilon, 1 + \varepsilon], \quad \varepsilon = 0.2$$

Terms:

- $\pi_{\text{new}}, \pi_{\text{old}}$ — current and rollout-time policies; ratio measures how much the policy changed
- ε — PPO clip width: updates outside $[0.8, 1.2]$ are clipped (gradient zeroed)

Meaning: when Alice keeps changing the goals, the ratio reflects the shifting goal distribution rather than Bob’s own update, so it lands outside the clip band ~80% of the time and the gradient is thrown away. Add to that sparse bonuses firing on 0.14% of pushes, and the advantage signal is just zero-mean noise.

Slide 26 — Why ASP Fails: Failure Mode 3 & Combined Effect

- Third, vanishing ABC gradient: ABC should provide a supervised signal when Bob fails — clone Alice’s trajectory. But Alice’s actions are high-entropy random walks because her entropy bonus (0.005×14 nats) dominates her surrogate loss (~0.005). The `bc_ratio` ≈ 1.0 for all trajectories, providing zero gradient direction
- The net effect: Bob receives no valid gradient from any of the three intended sources. His policy stays at random initialization forever. Evidence: Bob surrogate loss ≈ 0.001 for all 3000 iterations — the model isn’t even trying to learn

Formula breakdown

$$\text{bc_ratio} = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}}) \approx 1.0$$

Terms:

- `bc_ratio` — the imitation (behavioral-cloning) likelihood ratio between Bob’s current policy and Alice’s recorded actions

Meaning: if Alice acts almost randomly, every trajectory looks equally (im)probable to Bob, so the ratio is ≈ 1 and the cloning loss has no preferred direction — ABC supplies zero learning signal. Combined with modes 1 & 2, Bob gets no gradient from PPO, the environment, or ABC, and never leaves random initialisation.

Slide 27 — Key Result: PBRs Substitutes for Curriculum

- This is the central RQ3 contribution: a sufficiently informative reward can substitute for an explicit curriculum
- PPO-PBRs (no curriculum) reaches 80.0%; PPO-Curriculum reaches 76.7% — statistically tied (3.3pp, $p > 0.05$). At this batch size the staged curriculum adds no value
- Scope / honesty: the substitution is regime-dependent. At small PPO batch (gym, batch 960) the curriculum DOES help — PPO-Curriculum 50% vs PPO-PBRs 10%. The two interact through gradient variance, so I state the batch-size scope explicitly
- Self-play (ASP-dPose 6.7%, TASP-dPose 16.7%) helps in neither regime — but recall the confound: self-play changes ≥ 4 axes at once, so this is “self-play as implemented,” not an isolated test of the curriculum mechanism
- Practical implication: invest in reward design first — it is the higher-leverage lever; treat the training distribution as a separate diagnosis

Formula breakdown

PPO-PBRs 80.0% $\approx$ PPO-Curriculum 76.7% (n.s.); self-play 6.7–16.7% ($4.8\text{--}11.9 \times$ gap)

Meaning: at sufficient batch size a well-shaped reward and a staged curriculum reach the same place, so the reward substitutes for the curriculum; substituting self-play for the single agent collapses performance, but that package changes many things at once.

Slide 28 — Computational Overhead: Self-Play Is Not Slower

- Controlled, same-machine measurement: PPO-PBRs (single-agent) and the self-play variants run at the SAME per-iteration wall-clock (0.022 it/s) on the same 528-env GPU, with the same cuRobo IK + PhysX step — about $1.0\times$
- The 2-agent machinery (Alice + Bob rollouts, two PPO updates, GoalEncoder, ABC buffer, historical pool) adds no measurable per-iteration overhead, because the shared cuRobo IK + physics step dominates the cost
- I deliberately do NOT claim a 5–10 $\times$ self-play overhead — there is no controlled measurement supporting that number (it came from a different project/API, an apples-to-oranges comparison). The honest figure is $1.0\times$
- So the real cost of self-play is model/code complexity and added failure modes (two networks, non-stationary objective, ABC), not throughput — and the $4.8\times$ performance gap is not a compute-budget artifact, since both used the same compute

Slide 29 — Comparison Summary: All Models

- This is the unified comparison across all four evaluated models on the 30-scene protocol
- PPO-PBRs dominates: 80% validation, lowest errors, simplest architecture. PPO-Curriculum is statistically tied (76.7%)

- The self-play variants underperform: TASP-dPose 16.7%, ASP-dPose 6.7% — neither exceeds 10% on the combined pos+rot gate
- For RQ1 context, the hand-tuned classic dense reward is re-evaluated on the same protocol as the external comparator; PBRs is the recommended baseline

Slide 30 — Comparison Summary: Failure Taxonomy

- The comparison plot shows training curves over 3000 iterations. PPO-PBRs (blue) and PPO-Curriculum (orange) converge on similar trajectories; the self-play variants barely move
- The overarching pattern: every structural addition — curriculum controller, second agent, GoalEncoder, ABC buffer, historical pool, SE(2) metric — adds failure modes without adding performance. The simplest model wins

Slide 31 — What This Disproves

(no speaker notes yet)

Slide 32 — What This Supports & Disproves

- This slide summarizes the theoretical contributions — what the thesis supports and where claims must be scoped
- Claim 1 (Plappert): self-play generates an intrinsic curriculum for exploration. In our specific setting — contact-rich 3D pushing with cuRobo IK gating and a strict combined pos+rot gate, single-GPU budget — it did NOT yield a useful curriculum (6.7–16.7% vs 80%). This is N=1 task at one budget, so we scope it to this setting rather than refuting self-play in general
- Claim 2 (Sukhbaatar): the goal-encoder bottleneck helps Bob represent the goal. Our self-play runs fail at the exploration / training-distribution level, not the representation level — so representation tweaks cannot rescue it here (the base-ASP and encoder-ablation runs are excluded for lacking matched-protocol Isaac validation)
- Claim 3 (Narvekar/Portelas): curriculum learning helps when the base reward is informative. PPO-Curriculum’s first trigger was mis-specified (P82 bug); the corrected curriculum works (76.7%) but only ties PPO-PBRs — at this batch size a good reward substitutes for it
- The PBRs theory (Ng, Grzes) is strongly supported. The policy invariance theorem produced robust results — $k_p=30$, $k_r=5$, $w=10$ are not fragile hyperparameters; they work well across independent single-agent training runs

Formula breakdown

$$k_p = 30, \quad k_r = 5, \quad w = 10.0$$

Terms:

- k_p, k_r — position / rotation potential decay sharpness; w — potential weight (height)

Meaning: the same three potential constants worked across all single-agent runs and three independent seeds — evidence that PBRs is robust, not a fragile hand-tuned reward.

Slide 33 — Limitations & Research Gaps

(no speaker notes yet)

Slide 34 — Future Work: Promising Directions

- Our results are conclusive within the experimental budget, but they don't close the book on ASP. Several promising directions remain
- TASP-dPose-BobPenalty: The Bob time penalty creates a near-zero-sum game structure: Alice gains proportionally to $(t_B - t_A)$, Bob loses proportionally to t_B . From Letcher et al. (2019), the game Jacobian decomposes into symmetric (potential) and antisymmetric (Hamiltonian) components. Zero-sum games have pure antisymmetric dynamics, which are conservative — they cycle rather than diverge. This could stabilize self-play training where general-sum self-play diverges
- Time-based self-play + ABC: ABC with $\beta=0.5$ after Alice converges. Currently, ABC fails because Alice's actions are random, so the $bc_ratio \approx 1.0$ for all trajectories. If Alice converges first (e.g., with time-based reward and disabled Bob), then Bob starts with a meaningful imitation target
- Relaxed success gate: Bob's individual SR is 65-66%, but the combined gate almost never fires. Partial credit for achieving one of the two objectives could provide gradient to eventually learn the combined task
- PAIRED as an alternative: instead of Alice maximizing Bob's failure, the antagonist maximizes the protagonist's regret (difference between protagonist and antagonist returns). This produces solvable but challenging environments — addressing the exact failure mode we observed

Formula breakdown

$$R_B += -\gamma_{sp} t_B$$

Terms:

- R_B — Bob's reward; t_B — number of pushes Bob used; γ_{sp} — the same time-scale used for Alice

Meaning: penalising Bob for taking long makes Alice's gain $(+\gamma_{sp}(t_B - t_A))$ and Bob's loss $(-\gamma_{sp}t_B)$ nearly cancel — a near zero-sum game. Game theory (Letcher 2019) says zero-sum dynamics cycle rather than diverge, so this might stabilise the training that general-sum ASP destroys.

Slide 35 — Conclusion (1/2)

(no speaker notes yet)

Slide 36 — Conclusion (2/2)

- Three take-home messages

- First, PBRS works. PPO-PBRS achieves 80% on the hardest validation suite with the simplest architecture. This is a strong baseline that any future approach should be compared against. The theoretical justification (policy invariance, episodic validity, bounded smooth potentials) is fully supported by the empirical results
- Second, self-play does not pay off here. Both evaluated variants (outcome-based and time-based) underperform — 6.7% and 16.7% vs 80%. The failure sits in the non-stationary adversarial goal distribution, and is structural rather than a training-budget issue (identical per-iteration compute to the winning single-agent). But the self-play package changes many things at once, so we scope this to “self-play as implemented”
- Third, a sufficiently informative reward substitutes for an explicit curriculum (RQ3) — at sufficient batch size. This is the methodological contribution. Future work should: (a) design good rewards (PBRS works), (b) diagnose whether the training distribution supports learning, and (c) not assume that adding structure helps
- Practical recommendation: the simplest working system is the best starting point. Our 528-env PPO-PBRS trains in 1.6 days and achieves 80% SR. Start there, and only add complexity when you can prove it helps

Appendix

Slide 37 — Appendix Outline

(no speaker notes yet)

Slide 38 — Appendix A: Training Curves (All PBRS Models)

(no speaker notes yet)

Slide 39 — Appendix B: PBRS Theory Analysis

(no speaker notes yet)

Formula breakdown

$$\Phi(s) = w \exp(-k d^2) \qquad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

Terms:

- $\Phi(s)$ — the potential landscape; w — height, k — decay sharpness, d — distance to goal
- $c(s)$ — cosine angular distance; θ^* — goal yaw, θ — current yaw

Meaning: the appendix plots show Φ as a smooth bump peaking at the goal and compares its gradient with the old fractional formula; the cosine curve shows the rotation term is smooth and bounded across the full $[-\pi, \pi]$ range.